PROJETO FINAL – EMBARCATECH

Aluno: Alan Sovano Gomes

Mentora: Luana Stefany Moura dos Santos

Título: Malha Embarcada – uma plataforma para o estudo de sistemas de controle

1. Introdução

Dentre as diversas áreas de aplicações de sistemas embarcados, podemos elencar a área de sistemas de controle, a qual é frequentemente encontrada na grade curricular de cursos de engenharia (elétrica, biomédica, computação, mecatrônica, controle e automação, dentre outras). Um sistema de controle tem o objetivo de impor um determinado comportamento a uma variável de saída de um processo, a partir de uma variável de controle (Nise, 2012).

Podemos pensar em vários exemplos de sistemas de controle no dia a dia. Um ar-condicionado, por exemplo, mede a temperatura ambiente (variável cujo comportamento deseja-se impor) e, a partir dela, manipula a tensão de acionamento de seu compressor (variável de controle). Outro exemplo é o de um sujeito em uma unidade de tratamento intensivo (UTI) que está respirando através de um equipamento de ventilação mecânica: costuma-se definir, por exemplo, a pressão nas vias aéreas que é desejada na inspiração e expiração (variável que se deseja controlar), e o sistema entrega atua um conjunto de elementos mecânicos para fazer com que isso ocorra (sinais de controle).

Pensando em como tornar o projeto, a avaliação e o estudo de sistemas de controle mais didático, este trabalho apresenta o desenvolvimento de uma plataforma para o estudo prático e teórico de sistemas de controle, a qual foi intitulada de “Malha Embarcada”. O nome escolhido serve para chamar a atenção para o fato de que o sistema é inteiramente microprocessado, se tornado versátil e ideal para o teste de técnicas de controle digitais.

Para o desenvolvimento do projeto, foi desenvolvido um código em linguagem C para o microcontrolador RP2040, a base da placa Raspberry Pi Pico. Também foi desenvolvido um circuito eletrônico em placa de circuito impresso (PCI), o qual serve como sistema a ser controlado pelo microcontrolador. Para o funcionamento do sistema, foram utilizadas bibliotecas para: comunicação I2C com um display OLED; utilização de *Pulse Width Modulation* (PWM); conversão analógica digital (ADC); além de rotinas de interrupção e comunicação USB/UART para a coleta de dados pelo computador.

2. Objetivos

O objetivo deste projeto é construir uma plataforma embarcada para o teste e estudo de sistemas de controle em malha fechada. O sistema deve ser capaz de:

(I) Variar o valor da referência de entrada para o sistema;

(II) Alternar entre dois modos de funcionamento: modo em malha aberta (sem sistema de controle) e modo em malha fechada (com sistema de controle);

(III) O usuário deve ser capaz de mudar a referência ou o modo de operação a qualquer momento durante a execução do programa, conforme o pressionar de três botões distintos;

(IV) O usuário deve ser capaz de visualizar o sinal de saída do sistema e o sinal de controle através da comunicação USB/Serial, com o auxílio de um software especializado.

3. Justificativa

Apesar da extrema importância do ponto de vista prático, a área de sistemas de controle costuma ser de difícil entendimento por partes dos alunos de engenharia, o que ocorre devido ao alto grau de abstração da disciplina e à sofisticação das análises matemáticas que geralmente são necessárias para o projeto e avaliação de sistemas de controle em malha fechada (Chevalier *et al.*, 2021). Diante disso, atividades de natureza prática são essenciais para uma boa formação de engenheiros na área, procurando fornecer uma abordagem mais ativa no aprendizado da disciplina, o que costuma trazer bons resultados (Freitas *et al.*, 2022).

Cabe ressaltar, também, que sistemas microcontrolados são amplamente utilizados para a implementação de sistemas de controle digitais, especialmente em equipamentos mais modernos e sofisticados. Diante disso, a utilização de um sistema versátil e de custo relativamente baixo, como o Raspberry Pi Pico, se mostra uma boa opção para o cumprir os objetivos do projeto.

4. Descrição do hardware utilizado

A seguir, tanto a parte digital quanto a parte analógica do sistema proposto são descritas para o melhor entendimento do desenvolvimento do projeto.

(a) **Parte digital:** placa BitDogLab

O projeto foi inteiramente desenvolvido com base na placa BitDogLab, a qual é ilustrada na Figura 1. O botão A (conectado no GPIO 5) da placa foi utilizado para deixar o sistema no modo “malha aberta”, enquanto o botão B (conectado no GPIO 6) foi utilizado para por o sistema no modo “malha fechada”. Além disso, o botão do joystick (GPIO 22) foi utilizado para alternar o sinal de referência para o sistema de controle entre 0 (sem entrada) e 1 (entrada do tipo degrau unitário).

Figura 1 – Placa BitDogLab.

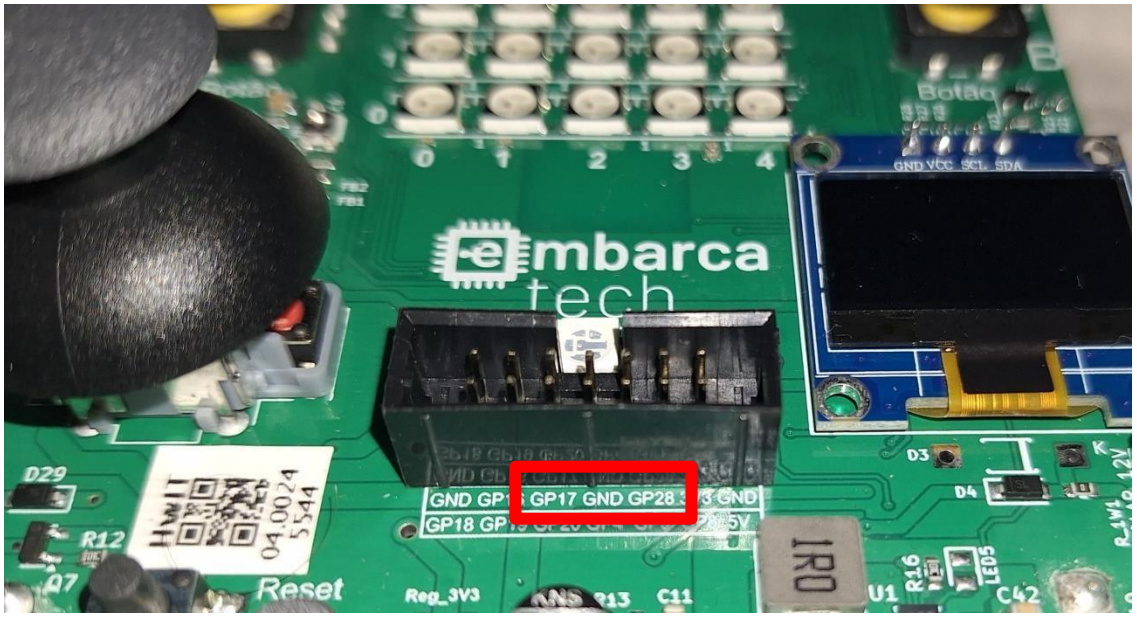
Fonte: Autor (2025).

Para utilizar os botões da placa, foi necessário configurar resistores internos de *pull-up* (o pressionar do botão envia um sinal em nível lógico baixo para o pino ao qual está ligado). Além disso, o GPIO 14 e o GPIO 15 foram utilizados para a comunicação I2C com o display OLED presente na placa.

Por fim, vale destacar a utilização do conversor A/D e do PWM. O conversor A/D é utilizado no sistema para medir a saída do circuito eletrônico construído (um sinal de tensão) a partir do GPIO 28. A partir da leitura desse sinal, o sinal de controle é calculado através de um algoritmo específico, gerando um valor de tensão que deve servir de entrada para o circuito eletrônico. Esse valor de tensão é enviado através do pino PWM configurado para tal tarefa (GPIO 17).

A escolha dos pinos PWM e do conversor A/D como GPIO 17 e GPIO 28 facilitaram a integração com a placa, visto que mesma possui esses pinos disponíveis junto a um ponto de referência (GND) em um conector de 14 pinos, conforme ilustra a Figura 2.

Figura 2 – Destaque para os pinos utilizados para a comunicação com o sistema eletrônico desenvolvido.

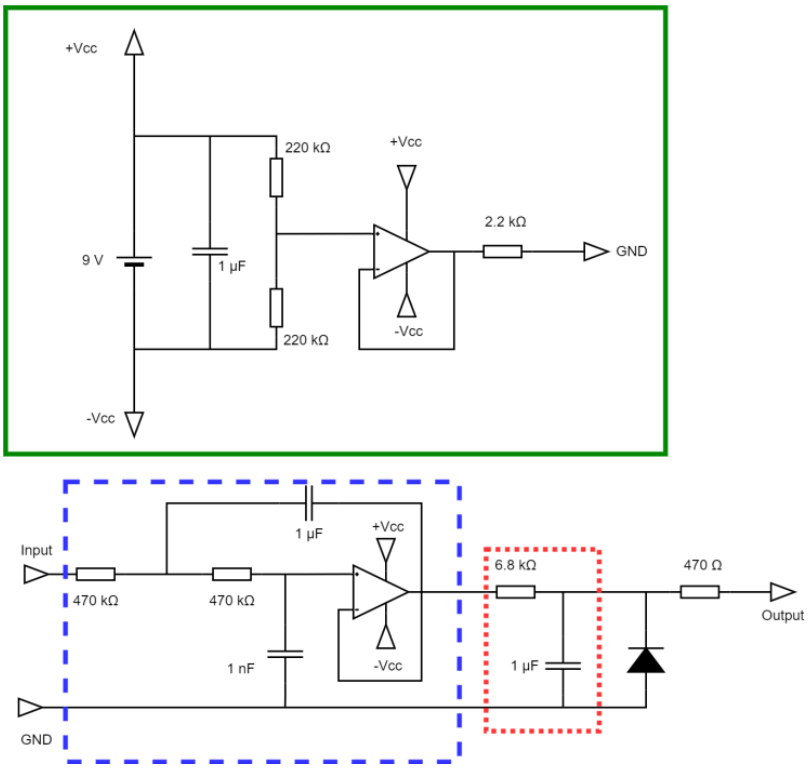


Fonte: Autor (2025).

(b) **Parte analógica:** circuito eletrônico oscilatório

O diagrama d circuito eletrônico desenvolvido pode ser visualizado na Figura 3. O circuito pode ser, basicamente, dividido em três partes: um gerador de fonte de tensão simétrica a partir de uma fonte de tensão não-simétrica (circuito destacado em verde); um filtro passa-baixas de 2ª ordem, do tipo Sallen-Key (destacado em azul); e um filtro passa-baixas de 1ª ordem (destacado em veremelho). O circuito conta, ainda, com um diodo de proteção para evitar que tensões abaixo de 0 V apareçam na saída do circuito (com o propósito de proteger o conversor A/D do microcontrolador).

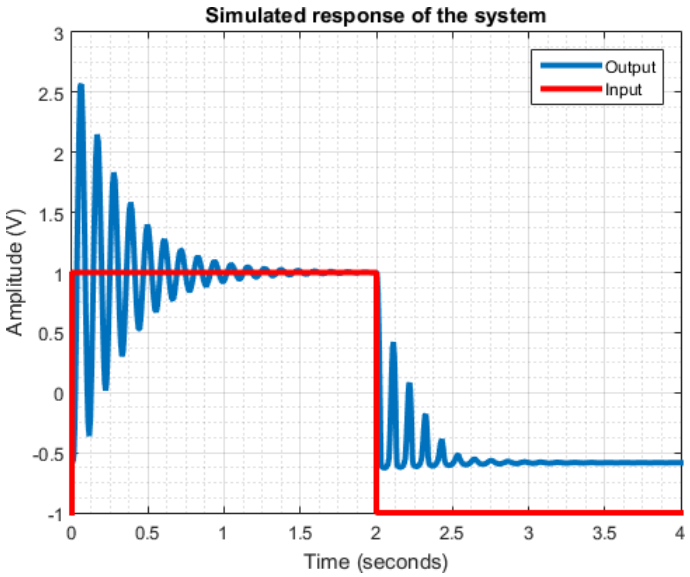
Figura 3 – Diagrama de circuito do sistema eletrônico desenvolvido.



Fonte: Autor (2025).

Os componentes dos filtros foram escolhidos para que o circuito tenha uma resposta oscilatória quando recebe uma resposta do tipo degrau (conforme ilustrado pela Figura 4). O circuito pode ser alimentado com uma bateria de 9 V ou com uma fonte de tensão de 7 V (para limitar a tensão máxima de saída). Sistemas oscilatórios costumam ser frequentemente estudados na teoria de sistemas de controle e, diante disso, o sistema a ser projetado para a planta deve cancelar ou diminuir tais oscilações.

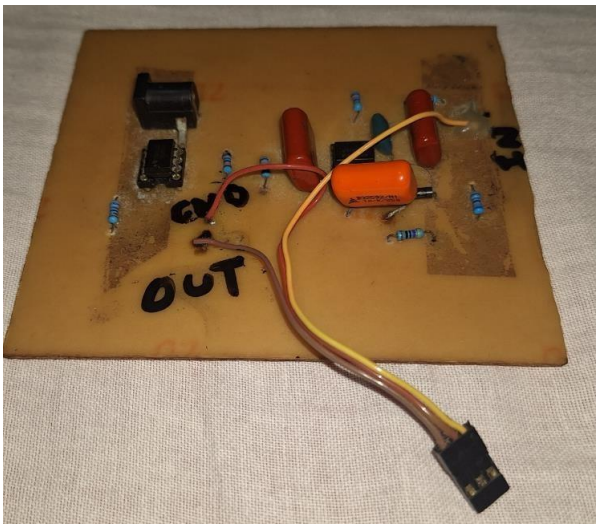
Figura 4 – Simulação da saída de tensão do circuito eletrônico desenvolvido diante de uma entrada do tipo degrau de 1 V e entrada do tipo degrau de -1 V.



Fonte: Autor (2025).

Por fim, a Figura 5 ilustra a placa de circuito impresso que foi de fato desenvolvida a partir do esquemático ilustrado pela Figura 3. Para que o sistema possa ser utilizado, há três pinos na placa: um de referência (GND), o qual deve ser conectado à referência do BitDogLab; um pino de entrada, o qual deve ser conectado ao PWM da placa BitDogLab; e um pino de saída, o qual deve ser conectado ao pino do conversor A/D da placa BitDogLab.

Figura 5 – Placa de circuito impresso confeccionada.



Fonte: Autor (2025).

(c) **Integração entre os sistemas**

Na Figura 6, pode-se visualizar como é fácil integrar o circuito analógico desenvolvido à placa BitDogLab, em uma abordagem do tipo *Plug and Play*. O propósito é facilitar ao máximo a montagem do *setup* experimental, de forma que a realização de experimentos envolvendo sistemas de controle seja atrativa.

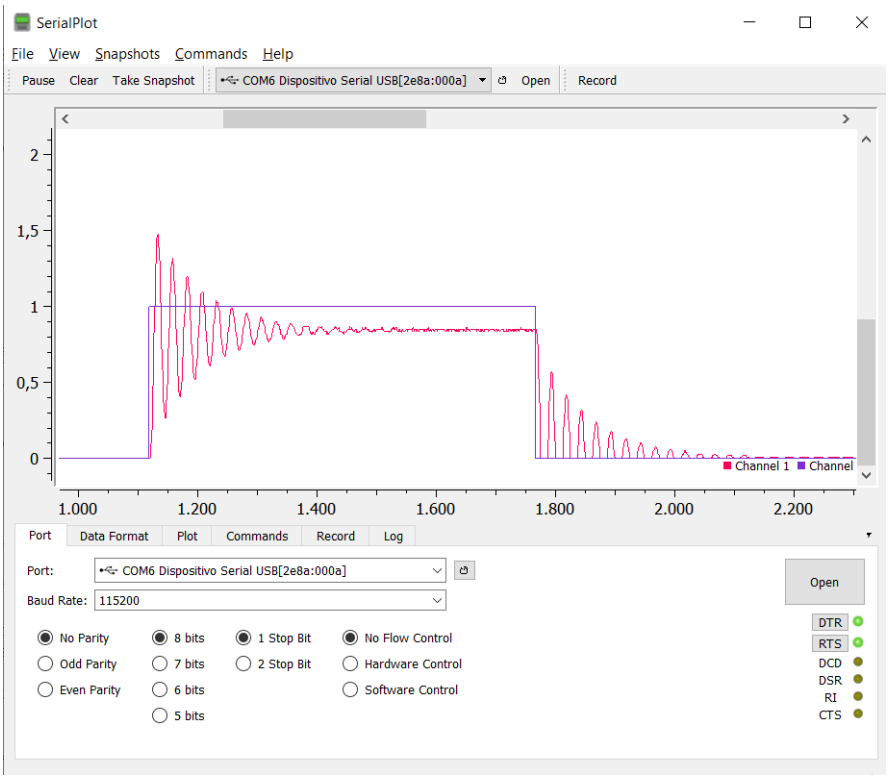
Figura 6 – Conexão entre a PCI desenvolvida e a BitDogLab.

Fonte: Autor (2025).

Por fim, foi utilizado um software livre denominado “SerialPlot”, o qual é comumente utilizado para plotar dados via comunicação serial. Para utilizá-lo, basta escolher a porta COM associada à comunicação USB/UART do BitDogLab, junto com o *baud rate* padrão da placa (definido como 115200 baud). O software permite, ainda, salvar os dados coletados no formato “.txt” para posterior análise (se desejado).

A Figura 7 ilustra a Tela do SerialPlot, o qual está sendo utilizado para o teste da placa em malha aberta. Vale ressaltar que, ainda na Figura 7, há um pequeno *offset* entre o sinal de saída desejado e a resposta do circuito eletrônico, o que não era previsto. Após investigação, percebeu-se que esse problema é decorrente da montagem da placa de circuito impresso. Isso não é um problema, pois o sistema de controle pode, também, eliminar *offsets* na saída de um processo (algo que, agora, também pode ser verificado).

Figura 7 – Tela do Software SerialPlot (dados recebidos pela placa BitDogLab).



Fonte: Autor (2025).

5. Descrição do software confeccionado

Para a confecção do software que opera o sistema, foram utilizadas as bibliotecas do *Software Development Kit* (SDK) do Raspberry Pi Pico, além de uma biblioteca externa para o controle do display OLED, intitulada “ssd1306.h”. Essa biblioteca vem com diferentes fontes e funções para escrever textos no display.

O software começa com uma rotina de inicialização, realizando a configuração dos pinos do microcontrolador RP2040 e mostrando algumas telas iniciais para o teste do display OLED, além de inicializar o sistema em malha aberta e com referência igual a 0 V. Em seguida, o sistema pede para o usuário escolher um modo de operação: modo “Malha aberta (A)” e modo “Malha fechada (B)”. A seleção do modo impõe qual o sinal de tensão enviado para o sistema eletrônico: se apenas uma referência bruta, do tipo degrau (0 V ou 1 V); ou se é enviado um sinal de controle apropriadamente calculado, a partir da referência, para que a saída não tenha um *offset* e não sofra oscilações. É possível variar a referência do tipo degrau entre 0 V e 1 V quantas vezes desejadas e em qualquer momento (estando o sistema em malha aberta ou em malha fechada).

Além do que foi descrito, o RP2040 também envia uma *string* pela comunicação USB/UART com o computador, a qual é composta pela saída do circuito (medida) e pelo

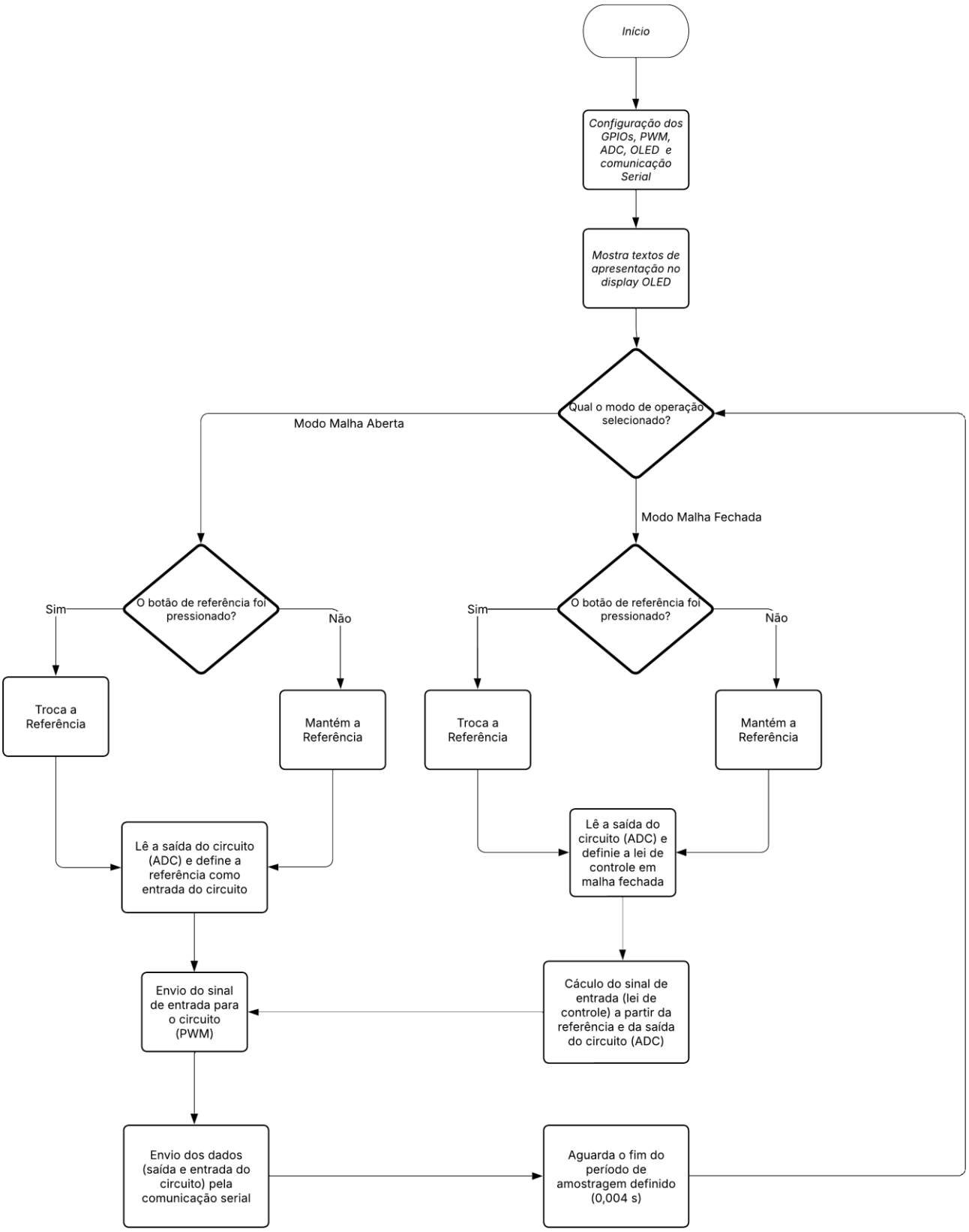
sinal de controle (calculado). Ambos os valores são extensivamente analisados em projetos e avaliações de sistemas de controle.

O funcionamento do programa descrito pode ser resumido, visualmente, através do fluxograma presente na Figura 8 (a qual é ilustrada somente na próxima página para sua melhor visualização).

Vale ressaltar que os envios dos dados da placa BitDogLab para o computador ocorrem em intervalos de 4 milissegundos (período de amostragem). Esse valor foi escolhido de acordo com o comportamento do sistema eletrônico confeccionado, de forma a evitar qualquer tipo de *aliasing* na representação dos sinais de controle e de saída.

O sistema de controle em malha fechada utilizado foi do tipo Proporcional-Integral-Derivativo (PID). O Controlador PID foi descrito em um formato digital, a partir de uma aproximação conhecida como aproximação *backward*. Julgou-se que os detalhes sobre o tipo de controlador e sua sintonia não caberiam neste relatório, mas optou-se por colocar tais informações caso haja um interesse por parte do leitor sobre o que foi feito. Em suma, o objetivo definido para o sistema de controle foi o de amortecer as oscilações e o de eliminar o *offset* na saída do sistema eletrônico para uma referência de valor constante.

Figura 8 – Fluxograma do software desenvolvido em linguagem C.



Fonte: Autor (2025).

6. Código

Os arquivos do projeto (bibliotecas externas, “CMakeLists.txt” e código principal) estão disponíveis no link a seguir, o qual leva a um repositório no GitHub. No repositório também está disponível um vídeo do projeto em funcionamento:

https://github.com/AlanSG7/Projetos_embarcados/tree/main/Projeto_final_embarcatech_capacitacao

A partir da próxima página, está descrito o código do arquivo principal feito para a aplicação. Em seguida, outro arquivo (referentes à biblioteca “ssd1306.h”) também é descrito (para que o código principal seja compreendido).

Vale ressaltar que mais três arquivos – “ssd1306.c”, “font.h” e “BMSPA_font.h” – também são necessários para o bom funcionamento do código. O primeiro é a implementação das funções descritas em “ssd1306.h”, enquanto os outros dois descrevem fontes de texto para serem utilizadas com o display OLED. Como, a partir do arquivo “ssd1306.h”, é possível entender o código principal (mesmo sem detalhes, sobre “ssd1306.c”), optou-se por colocar somente esse código extra neste documento. Entretanto, deve-se ressaltar, mais uma vez, que todos os outros códigos secundários estão no link fornecido anteriormente. O repositório original da biblioteca utilizada pode ser encontrado no link a seguir:

https://github.com/daschr/pico-ssd1306/tree/main

CÓDIGO PRINCIPAL

```
/*PROJETO FINAL - EMBARCATECH
Aluno: Alan Sovano Gomes
Mentora: Luana Stefany Moura dos Santos
Tema: Malha Embarcada -- uma plataforma para o estudo de sistemas de controle
*/


// Bibliotecas padrão do raspberry pi pico SDK
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "pico/stdlib.h"
#include "hardware/i2c.h"
#include "hardware/pwm.h"
#include "hardware/adc.h"

// Bibliotecas utilizadas para trabalhar com o display OLED
#include "ssd1306.h"
#include "BMSPA_font.h"

// Defines para os botões, pino PWM e conversor AD
#define PUSH_BUTTON_A 5
#define PUSH_BUTTON_B 6
#define PUSH_BUTTON_X 22
#define PWM_PIN 17
#define ADC_LEITURA 2

/**********************Declaração de variáveis***************************/

// Variáveis de configuração do PWM.
// f_PWM = 125*(10^6)/((5)*(PERIOD+1))
// Se PERIOD = 2499 -> f_PWM = 10 kHZ; Se PERIOD = 24999 -> f_PWM = 1 kHz
const uint16_t PERIOD_PWM =24999;
const float DIVIDER_PWM = 5;
volatile uint16_t pwm_level = (PERIOD_PWM+1);

// Fonte sendo utilizada
const uint8_t *fonts= {BMSPA_font};

// Variáveis para controlar o modo de oeração do sistema e mostrar no display
OLED
volatile absolute_time_t ultima_pressao_A = 0, ultima_pressao_B = 0,
ultima_pressao_X = 0; // Armazena o tempo da última pressão dos botões.
const char *words_2[]= {"MODO SELECIONADO:","Malha aberta", "Malha fechada",
"(PID digital)", "(Sem controlador)"};
volatile bool degrau = false;
volatile bool malha_fechada = false;
```

```
// Variável para armazenar a referência atual
volatile float yr = 0.0;

 // Parâmetros do controlador contínuo (PID)
const float Kp = 0.001;
const float Ki = 4.0;
const float Kd = 0.0001;

// Passando o controlador contínuo para o domínio discreto através de uma
aproximação backward (gerando um PID digital)
const float Ts = 0.004;
const float s0 = Kp + Ki*Ts + Kd/Ts;
const float s1 = -Kp - 2*Kd/Ts;
const float s2 = Kd/Ts;


// Condições iniciais do sistema de controle
volatile float y = 0;
volatile float u[2] = {0.0, 0.0};
volatile float e[3] = {0.0, 0.0, 0.0};
volatile float u_pwm =0.0;

// Criando variável para configuração do temporizador repetitivo
struct repeating_timer temporizador_controle;


/*************************declaração de funções *************************/

// Função de inicialização do PWM
void setup_pwm()
{

  // Variáveis de slice
  uint slice_pwm;

  // Setando pinos de PWM
  gpio_set_function(PWM_PIN, GPIO_FUNC_PWM);

  // Obtendo valor dos slices
  slice_pwm = pwm_gpio_to_slice_num(PWM_PIN);

  // Definindo divisor de clock do PWM
  pwm_set_clkdiv(slice_pwm, DIVIDER_PWM);

  // Valor do contador do PWM (período)
  pwm_set_wrap(slice_pwm, PERIOD_PWM);

  // Habilitando o PWM no slice correspondente
  pwm_set_enabled(slice_pwm, true);
```

```
  // Enviando valor inicial (0) para a porta pwm
  pwm_set_gpio_level(PWM_PIN, 0*pwm_level);


}


// Inicializando pinos de entrada e saída
void setup_gpios() {

  //Comunicação         I2C
    i2c_init(i2c1, 400000);
    gpio_set_function(14, GPIO_FUNC_I2C);
    gpio_set_function(15, GPIO_FUNC_I2C);
    gpio_pull_up(14);
    gpio_pull_up(15);

  // Inicializando o pino do botão A
  gpio_init(PUSH_BUTTON_A);
  gpio_set_dir(PUSH_BUTTON_A, GPIO_IN);
  gpio_pull_up(PUSH_BUTTON_A);

   // Inicializando o pino do botão B
  gpio_init(PUSH_BUTTON_B);
  gpio_set_dir(PUSH_BUTTON_B, GPIO_IN);
  gpio_pull_up(PUSH_BUTTON_B);

   // Inicializando o pino do botão X
  gpio_init(PUSH_BUTTON_X);
  gpio_set_dir(PUSH_BUTTON_X, GPIO_IN);
  gpio_pull_up(PUSH_BUTTON_X);


}


// Animação inicial do programa
void animacao_inicial(void) {

  // Vetor com as strings iniciais que serão mostradas
  const char *words[]= {"MALHA EMBARCADA", "UM PROJETO" , "EMBARCATECH",
"SELECIONE O MODO:", "Malha Aberta (A)", "Malha fechada (B)"};

  // Declarando variável utilizada para definir o display OLED
  ssd1306_t disp;
  disp.external_vcc=false;
  ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
  ssd1306_clear(&disp);
```

```
  // Primeira tela
  ssd1306_draw_string_with_font(&disp, 5, 30, 1, fonts, words[0]);
  ssd1306_show(&disp);
  sleep_ms(2500);
  ssd1306_clear(&disp);

  // Segunda tela
  ssd1306_draw_string_with_font(&disp, 18, 20, 1, fonts, words[1]);
  ssd1306_draw_string_with_font(&disp, 15, 35, 1, fonts, words[2]);
  ssd1306_show(&disp);
  sleep_ms(2500);
  ssd1306_clear(&disp);

  // Terceira tela
  ssd1306_draw_string(&disp, 0, 20, 1, words[3]);
  ssd1306_draw_string(&disp, 0, 35, 1, words[4]);
  ssd1306_draw_string(&disp, 0, 45, 1, words[5]);
  ssd1306_show(&disp);
  ssd1306_clear(&disp);

}

// Função para conversão do valor lido pelo conversor A/D
float adc_to_voltage(uint16_t adc_value)
{
  // Constantes fornecidas no datasheet do RP2040
  const float conversion_factor = 3.3f / (1 << 12);  // Conversão de 12 bits
(0-4095) para 0-3.3V
  float voltage = adc_value * conversion_factor;     // Converte o valor ADC
para tensão
  return voltage;
}



// Função para tratar a interrupção dos botões
void mudar_operacao_callback(int gpio, uint32_t events)
{

  //Declarando uma variável struct que irá auxiliar na mudança dos dizeres
presentes no display OLED
  ssd1306_t disp;
  disp.external_vcc=false;
  ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
  ssd1306_clear(&disp);
```

```
  // Verifica se a interrupção foi disparada pelo botão A (modo malha aberta)
  if(gpio==PUSH_BUTTON_A && events == GPIO_IRQ_EDGE_FALL &&
absolute_time_diff_us(ultima_pressao_A, get_absolute_time()) > 500000)
  {
    ultima_pressao_A = get_absolute_time();
    ssd1306_draw_string(&disp,  15,  20,  1,  words_2[0]);
    ssd1306_draw_string(&disp,  15,  35,  1,  words_2[1]);
    ssd1306_draw_string(&disp,  15,  50,  1,  words_2[4]);
    ssd1306_show(&disp);
    ssd1306_clear(&disp);
    malha_fechada = false;

  }

  // Verifica se a interrupção foi disparada pelo botão B (modo malha fechada)
  else if(gpio==PUSH_BUTTON_B && events == GPIO_IRQ_EDGE_FALL &&
absolute_time_diff_us(ultima_pressao_B, get_absolute_time()) > 500000)
  {
    ultima_pressao_B = get_absolute_time();
    ssd1306_draw_string(&disp,  15,  20,  1,  words_2[0]);
    ssd1306_draw_string(&disp,  15,  35,  1,  words_2[2]);
    ssd1306_draw_string(&disp,  15,  50,  1,  words_2[3]);
    ssd1306_show(&disp);
    ssd1306_clear(&disp);
    malha_fechada = true;

  }

  // Verifica qual o valor de referência para o degrau (0 ou 1)
  else if(gpio==PUSH_BUTTON_X && events == GPIO_IRQ_EDGE_FALL &&
absolute_time_diff_us(ultima_pressao_X, get_absolute_time()) > 500000)
  {
    ultima_pressao_X = get_absolute_time();
    degrau = !degrau;

    if(degrau==false)
    {
      yr = 0.0;
    }
    else
    {
      yr = 1.0;
    }

  }

}
```

```
// Laço com o algoritmo de controle (o qual deve rodar no período de
amostragem de 4 ms)
bool controle_temporizador_callback(struct repeating_timer *t)
{
 // Calculando o erro de seguimento de referência e o sinal de controle

  if(malha_fechada==true)
  {
    // Lendo sinal de saída do sistema
    uint16_t adc_value = adc_read();
    // Convertendo os valores lidos pelo ADC para tensão
    y = adc_to_voltage(adc_value);

    // Calculando sinal de erro e sinal de controle
    e[0] = yr - y;
    u[0] = u[1] +s0*e[0] +s1*e[1] +s2*e[2];

    // Saturando o sinal de controle
    if(u[0]<0)
    {
      u[0]=0;
    }

    if(u[0]>3.3)
    {
      u[0]=3.3;
    }

    // Calculando nível do PWM referente ao sinal de controle
    u_pwm = 303*u[0]/1000;

    // Atualizando os erros e os sinais de controle
    e[2] = e[1];
    e[1] = e[0];
    u[1] = u[0];

    // Enviando o sinal de controle para o sistema a ser controlado
    pwm_set_gpio_level(PWM_PIN, u_pwm*pwm_level);

    // Enviando os dados pela porta USB
    printf("%.2f,%.2f\n",y,u[0]);

  }
  else
  {
    // Sinal de saída sendo lido e convertido para volts
    uint16_t adc_value = adc_read();
    y = adc_to_voltage(adc_value);

    // Enviando a referência para o sistema (malha aberta)
```

```
    u[0] = yr;

    // Atualizando o sinal de entrada
    u[1] = u[0];

    // Enviando o sinal de controle para o sistema
    u_pwm = 303*u[0]/1000;
    pwm_set_gpio_level(PWM_PIN, u_pwm*pwm_level);

    // Enviando o sinal de saída e de controle para a comunicação serial
    printf("%.2f,%.2f\n",y, u[0]);

  }

}


/****************************Função main*********************************/
int main()
{
  // Inicializando as entradas e saída padrões
  stdio_init_all();

  // Configurando pinos de entrada e saída para a comunicação I2C
  setup_gpios();

  // Animação inicial
  animacao_inicial();

  //inicializando PWM
  setup_pwm();

  // inicializando ADC
  adc_init();
  adc_gpio_init(28); // Garantindo que o pino 28 (ADC) está em alta impedância
(sem pull_up ou pull_down)
  adc_select_input(ADC_LEITURA);

  // Configurando a interrupção gerada pelo botão A, B e X
  gpio_set_irq_enabled_with_callback(PUSH_BUTTON_A, GPIO_IRQ_EDGE_FALL, true,
mudar_operacao_callback);
  gpio_set_irq_enabled_with_callback(PUSH_BUTTON_B, GPIO_IRQ_EDGE_FALL, true,
mudar_operacao_callback);
  gpio_set_irq_enabled_with_callback(PUSH_BUTTON_X, GPIO_IRQ_EDGE_FALL, true,
mudar_operacao_callback);

  // Cria o evento repetitivo. calcula e envia o sinal de controle a cada Ts
períodos de tempo
  add_repeating_timer_ms(4, controle_temporizador_callback, NULL,
&temporizador_controle);
```

```
  // Loop infinito
  while(true)
  {
    tight_loop_contents(); // Função que otimiza o loop vazio para evitar
consumo excessivo de CPU.
  }


  return 0;

}
```

ARQUIVO “ssd1306.h”

```
/*
MIT License

Copyright (c) 2021 David Schramm

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
*/

/**
* @file ssd1306.h
*
* simple driver for ssd1306 displays
*/

#ifndef _inc_ssd1306
#define _inc_ssd1306
#include <pico/stdlib.h>
#include <hardware/i2c.h>

/**
*   @brief defines commands used in ssd1306
*/
typedef enum {
    SET_CONTRAST = 0x81,
    SET_ENTIRE_ON = 0xA4,
    SET_NORM_INV = 0xA6,
    SET_DISP = 0xAE,
    SET_MEM_ADDR = 0x20,
    SET_COL_ADDR = 0x21,
    SET_PAGE_ADDR = 0x22,
```

```
    SET_DISP_START_LINE = 0x40,
    SET_SEG_REMAP = 0xA0,
    SET_MUX_RATIO = 0xA8,
    SET_COM_OUT_DIR = 0xC0,
    SET_DISP_OFFSET = 0xD3,
    SET_COM_PIN_CFG = 0xDA,
    SET_DISP_CLK_DIV = 0xD5,
    SET_PRECHARGE = 0xD9,
    SET_VCOM_DESEL = 0xDB,
    SET_CHARGE_PUMP = 0x8D
} ssd1306_command_t;

/**
*   @brief holds the configuration
*/
typedef struct {
    uint8_t width;      /**< width of display */
    uint8_t height;     /**< height of display */
    uint8_t pages;      /**< stores pages of display (calculated on
initialization*/
    uint8_t address;    /**< i2c address of display*/
    i2c_inst_t *i2c_i;  /**< i2c connection instance */
    bool external_vcc;  /**< whether display uses external vcc */
    uint8_t *buffer;    /**< display buffer */
    size_t bufsize;     /**< buffer size */
} ssd1306_t;

/**
*   @brief initialize display
*
*   @param[in] p : pointer to instance of ssd1306_t
*   @param[in] width : width of display
*   @param[in] height : heigth of display
*   @param[in] address : i2c address of display
*   @param[in] i2c_instance : instance of i2c connection
*
*   @return bool.
*   @retval true for Success
*   @retval false if initialization failed
*/
bool ssd1306_init(ssd1306_t *p, uint16_t width, uint16_t height, uint8_t
address, i2c_inst_t *i2c_instance);

/**
*   @brief deinitialize display
*
*   @param[in] p : instance of display
*
*/
void ssd1306_deinit(ssd1306_t *p);
```

```c
/**
*   @brief turn off display
*
*   @param[in] p : instance of display
*
*/
void ssd1306_poweroff(ssd1306_t *p);

/**
    @brief turn on display

    @param[in] p : instance of display

*/
void ssd1306_poweron(ssd1306_t *p);

/**
    @brief set contrast of display

    @param[in] p : instance of display
    @param[in] val : contrast

*/
void ssd1306_contrast(ssd1306_t *p, uint8_t val);

/**
    @brief set invert display

    @param[in] p : instance of display
    @param[in] inv : inv==0: disable inverting, inv!=0: invert

*/
void ssd1306_invert(ssd1306_t *p, uint8_t inv);

/**
    @brief display buffer, should be called on change

    @param[in] p : instance of display

*/
void ssd1306_show(ssd1306_t *p);

/**
    @brief clear display buffer

    @param[in] p : instance of display

*/
void ssd1306_clear(ssd1306_t *p);
```

```
/**
    @brief clear pixel on buffer

    @param[in] p : instance of display
    @param[in] x : x position
    @param[in] y : y position
*/
void ssd1306_clear_pixel(ssd1306_t *p, uint32_t x, uint32_t y);

/**
    @brief draw pixel on buffer

    @param[in] p : instance of display
    @param[in] x : x position
    @param[in] y : y position
*/
void ssd1306_draw_pixel(ssd1306_t *p, uint32_t x, uint32_t y);

/**
    @brief draw line on buffer

    @param[in] p : instance of display
    @param[in] x1 : x position of starting point
    @param[in] y1 : y position of starting point
    @param[in] x2 : x position of end point
    @param[in] y2 : y position of end point
*/
void ssd1306_draw_line(ssd1306_t *p, int32_t x1, int32_t y1, int32_t x2,
int32_t y2);

/**
    @brief clear square at given position with given size

    @param[in] p : instance of display
    @param[in] x : x position of starting point
    @param[in] y : y position of starting point
    @param[in] width : width of square
    @param[in] height : height of square
*/
void ssd1306_clear_square(ssd1306_t *p, uint32_t x, uint32_t y, uint32_t
width, uint32_t height);

/**
    @brief draw filled square at given position with given size

    @param[in] p : instance of display
    @param[in] x : x position of starting point
    @param[in] y : y position of starting point
    @param[in] width : width of square
```

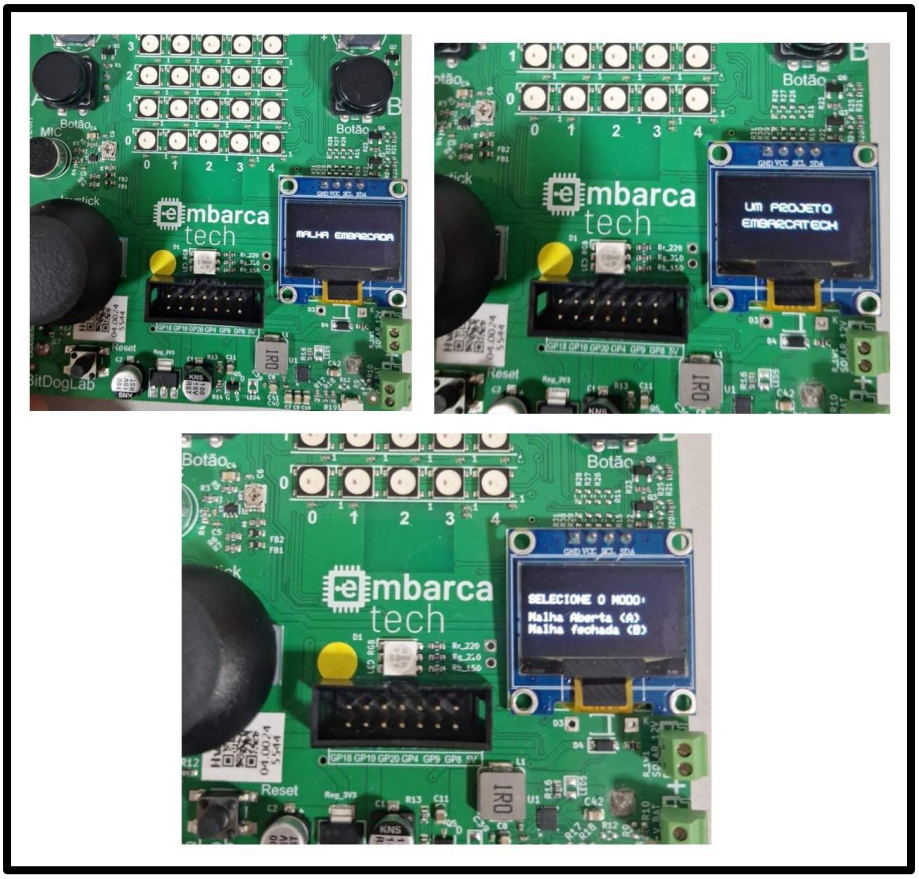
```
    @param[in] height : height of square
*/
void ssd1306_draw_square(ssd1306_t *p, uint32_t x, uint32_t y, uint32_t width,
uint32_t height);

/**
    @brief draw empty square at given position with given size

    @param[in] p : instance of display
    @param[in] x : x position of starting point
    @param[in] y : y position of starting point
    @param[in] width : width of square
    @param[in] height : height of square
*/
void ssd1306_draw_empty_square(ssd1306_t *p, uint32_t x, uint32_t y, uint32_t
width, uint32_t height);

/**
    @brief draw monochrome bitmap with offset

    @param[in] p : instance of display
    @param[in] data : image data (whole file)
    @param[in] size : size of image data in bytes
    @param[in] x_offset : offset of horizontal coordinate
    @param[in] y_offset : offset of vertical coordinate
*/
void ssd1306_bmp_show_image_with_offset(ssd1306_t *p, const uint8_t *data,
const long size, uint32_t x_offset, uint32_t y_offset);

/**
    @brief draw monochrome bitmap

    @param[in] p : instance of display
    @param[in] data : image data (whole file)
    @param[in] size : size of image data in bytes
*/
void ssd1306_bmp_show_image(ssd1306_t *p, const uint8_t *data, const long
size);

/**
    @brief draw char with given font

    @param[in] p : instance of display
    @param[in] x : x starting position of char
    @param[in] y : y starting position of char
    @param[in] scale : scale font to n times of original size (default should
be 1)
    @param[in] font : pointer to font
    @param[in] c : character to draw
*/
```

```
void ssd1306_draw_char_with_font(ssd1306_t *p, uint32_t x, uint32_t y,
uint32_t scale, const uint8_t *font, char c);

/**
    @brief draw char with builtin font

    @param[in] p : instance of display
    @param[in] x : x starting position of char
    @param[in] y : y starting position of char
    @param[in] scale : scale font to n times of original size (default should
be 1)
    @param[in] c : character to draw
*/
void ssd1306_draw_char(ssd1306_t *p, uint32_t x, uint32_t y, uint32_t scale,
char c);

/**
    @brief draw string with given font

    @param[in] p : instance of display
    @param[in] x : x starting position of text
    @param[in] y : y starting position of text
    @param[in] scale : scale font to n times of original size (default should
be 1)
    @param[in] font : pointer to font
    @param[in] s : text to draw
*/
void ssd1306_draw_string_with_font(ssd1306_t *p, uint32_t x, uint32_t y,
uint32_t scale, const uint8_t *font, const char *s );

/**
    @brief draw string with builtin font

    @param[in] p : instance of display
    @param[in] x : x starting position of text
    @param[in] y : y starting position of text
    @param[in] scale : scale font to n times of original size (default should
be 1)
    @param[in] s : text to draw
*/
void ssd1306_draw_string(ssd1306_t *p, uint32_t x, uint32_t y, uint32_t scale,
const char *s);

#endif
```

7. Resultados

Após a implementação e execução do código, os resultados principais puderam ser registrados. As telas iniciais mostradas no display OLED podem ser visualizadas na Figura 9, com os dizeres: “Malha Embarcada”, seguido de “Um projeto embarcatech” e, por fim, seguido pela tela inicial de seleção de modo.

Figura 9 – Telas de inicialização da aplicação “Malha Embarcada”.



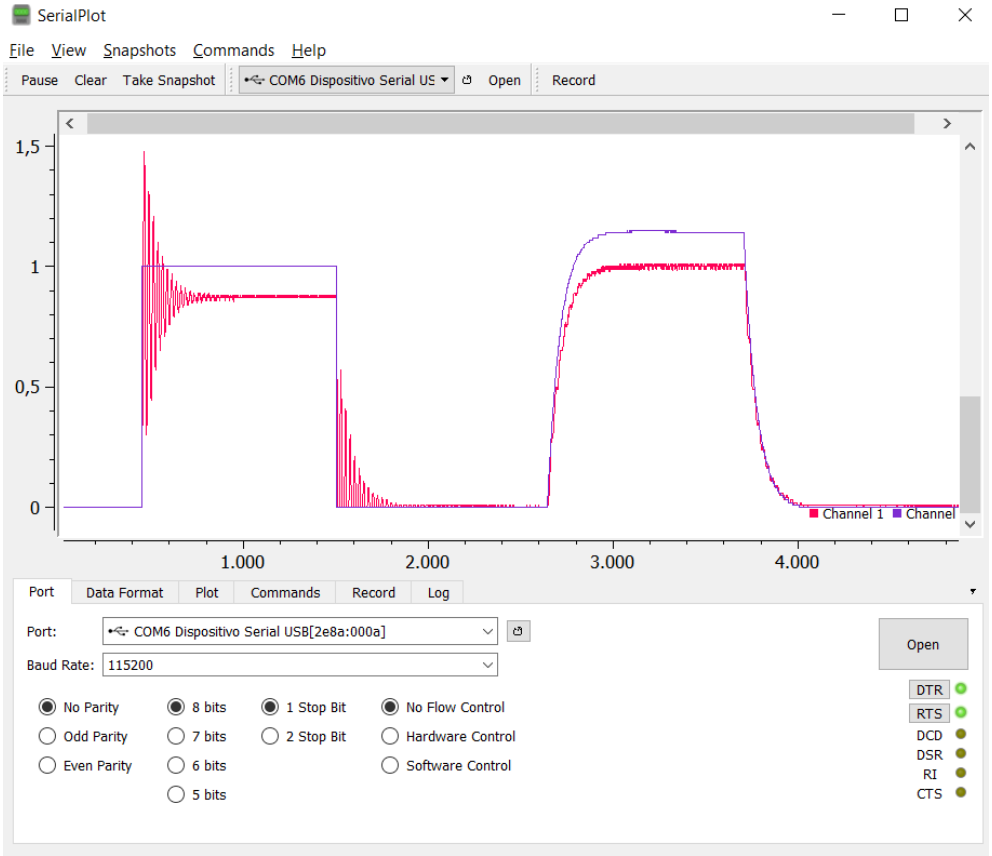
Fonte: Autor (2025).

Depois da seleção de modo (se pressionado o botão A ou o botão B), o display OLED mostra ao usuário qual foi o modo de operação selecionado, conforme é possível visualizar na Figura 10. Por fim, a Figura 11 ilustra a saída do sistema eletrônico com o controlador desligado e com o controlador ligado. Em cada caso, ocorreu a variação do sinal de referência (de 0 V para 1 V e de 1 V para 0 V) com o apertar do botão associado ao *joystick* da placa BitDogLab.

Figura 10 – Telas da aplicação “Malha Embarcada” após a seleção de um dos modos de operação.

Fonte: Autor (2025).

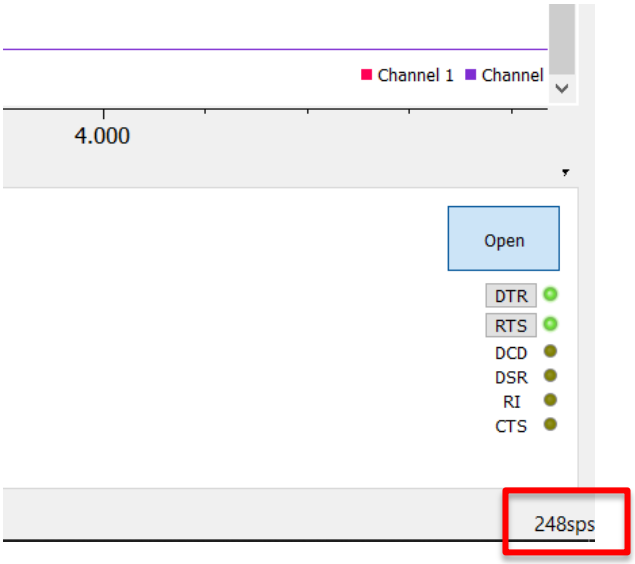
Figura 11 – Saída do circuito no software SerialPlot para o sistema de controle ligado e desligado. Em cada caso, a saída foi variada de 0 V para 1 V e, em seguida, para 0 V novamente.



Fonte: Autor (2025).

Por fim, cabe ressaltar que o software conseguiu realizar as tarefas necessárias dentro do tempo de amostragem de aproximadamente 4 milissegundos, conforme definido. No software SerialPlot, conforme ilustrado pela Figura 12, é possível visualizar a taxa de amostragem do sistema (o qual indicou que era de 250 Hz, ou 4 milissegundos).

Figura 12 – Taxa de amostragem calculada pelo software SerialPlot em hertz ou amostras por segundo (*samples per second*).



Fonte: Autor (2025).

8. Considerações finais

É possível constatar, de acordo com os resultados obtidos, que os objetivos definidos para a conclusão inicial do projeto foram alcançados. Pretende-se continuar o projeto em momentos futuros, adicionando novos tipos de controladores e a opção de variar a entrada do tipo degrau em intervalos maiores.

Durante o desenvolvimento do projeto, surgiu a ideia de também acoplar, futuramente, mais um display OLED para mostrar os gráficos do sinal de saída e de controle. Dessa forma, seria possível ter um sistema mais independente do computador, deixando o sistema portátil, o que seria útil para aplicações em sala de aula e laboratórios de ensino de sistemas de controle.

Por fim, vale destacar que a comunicação com o display OLED foi a principal dificuldade encontrada ao longo do projeto. A confecção de uma biblioteca própria para a comunicação com o dispositivo pode ser interessante em longo prazo, visando um entendimento mais detalhado sobre como o dispositivo funciona em termos de memória e registradores internos.

9. Referências

Chevalier, A., Dekemele, K., Juchem, J., & Loccufier, M. **Student feedback on educational innovation in control engineering: active learning in practice.** IEEE Transactions on Education, v. 64, n. 4, p. 432-437, 2021.

Freitas, E. J., Prado, L. S., Silva, M. V., Alvarenga, V. A., & Santana, A. C. **Active Learning Strategy Applied to Control Theory Teaching.** International Journal of Advanced Engineering Research and Science, v. 9, p. 9, 2022.

NISE, Norman S. Engenharia de Sistemas de Controle. 6a Ediçao. 2012.